

Beyond the Syllabus

Is Quadratic Really Necessary? Fine-Grained Lower Bounds and the Fall of Bellman–Ford's
70-Year Barrier

Week 9 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved two upper bounds: edit distance / LCS in $O(nm)$ time via a 2-D DP table, and shortest paths with negative edges in $O(mn)$ time via Bellman–Ford.
- Two questions a curious student should ask next:
 - ① Is $O(nm)$ edit distance actually the **best possible**, or is a truly-subquadratic algorithm just waiting to be discovered?
 - ② Dijkstra beats brute-force shortest paths when weights are non-negative. Is there an analogous “fast” algorithm once negative weights are allowed, or is $O(mn)$ Bellman–Ford as good as it gets?
- The first question was answered (no) in 2015. The second was answered (yes!) only in 2022 – and sharpened again in **2026**.

How this supplement is different from a survey

Every result below is a specific, citable paper (author, venue, year, and an arXiv/DOI identifier), verified before this slide was written – not a vague “researchers have shown.”

Recap: the bound you proved this week

This week's lecture

Edit distance / sequence alignment between strings of length n and m is computable in $O(nm)$ time via a 2-D dynamic-programming table, using the “what happens to the last character” recurrence.

- Every algorithms textbook presents $O(nm)$ as *the* answer – but is it optimal, or just the best anyone happened to find?
- Ruling out something faster needs a completely different kind of argument: a **conditional lower bound**, tying edit distance's difficulty to a widely-believed complexity-theoretic conjecture.

The conjecture: SETH

Strong Exponential Time Hypothesis (SETH)

Boolean satisfiability on n variables cannot be solved in $O(2^{(1-\varepsilon)n})$ time for *any* fixed $\varepsilon > 0$ – i.e. no algorithm beats exhaustive search by an exponentially-growing factor.

- SETH is believed true (decades of SAT-solving research have found no counterexample) but is *unproven* – exactly like $P \neq NP$.
- A “SETH-hardness” result for problem X says: *if* X has a truly-subquadratic algorithm, *then* SETH is false. So a faster algorithm for X would be a major breakthrough, not just an incremental improvement.
- The standard tool for proving this is a fine-grained reduction from **Orthogonal Vectors (OV)**: given two sets of n boolean vectors in $\{0, 1\}^d$ ($d = O(\log n)$), decide if some pair is orthogonal. OV is known to require $n^{2-o(1)}$ time under SETH.

The result: edit distance is SETH-hard

Backurs & Indyk, STOC 2015

“Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false).”

DOI: 10.1145/2746539.2746612.

- They give a reduction turning an Orthogonal Vectors instance into two strings X, Y of length $O(n)$ such that $\text{edit_distance}(X, Y)$ reveals whether an orthogonal pair exists.
- **Consequence:** an $O((nm)^{1-\varepsilon})$ edit-distance algorithm, for any $\varepsilon > 0$, would give a truly-subquadratic OV algorithm, hence refute SETH.
- So the $O(nm)$ bound from this week's lecture is, modulo SETH, **exactly optimal** – not a limitation of Kleinberg–Tardos's presentation, but a genuine wall.

What the lower bound does *not* rule out

- The Backurs–Indyk bound is about **exact**, **classical** algorithms only. Two escape hatches remain, and both are real, active research:
 - ① **Approximation.** Allowing a constant-factor approximation to the edit distance (rather than the exact value) sidesteps the reduction entirely.
 - ② **Quantum computation.** A quantum algorithm might exploit structure the SETH reduction doesn't forbid.
- A 2018 result combines *both* escape hatches at once.

The escape hatch: a quantum algorithm beats the wall

Boroujeni, Ehsani, Ghodsi, HajiAghayi & Seddighin, SODA 2018

“Approximating Edit Distance in Truly Subquadratic Time: Quantum and MapReduce.”
arXiv: 1804.04178.

- They give a **quantum** algorithm running in $O(n^{1.858})$ time that approximates edit distance within a factor of 7 – genuinely subquadratic, and provably impossible *classically* (exactly) under SETH.
- A second variant pushes the exponent down to $O(n^{1.781})$ for a larger approximation factor – a direct time/accuracy trade-off.
- **Why this doesn't contradict Backurs–Indyk:** the SETH-hardness proof is for the *exact* value; approximating within a constant factor is a genuinely easier problem, and quantum algorithms are a genuinely different computational model.

Section takeaway

- 1 This week's $O(nm)$ DP for *exact* edit distance is, modulo a widely-believed conjecture (SETH), the best any classical algorithm can ever do.
- 2 That is a remarkably strong and clean statement – most algorithms you meet in this course have *no* matching lower bound at all.
- 3 The wall is specifically about exact, classical computation: relax either constraint (allow approximation, allow quantum resources) and truly-subquadratic algorithms exist today.

Recap: the bound you proved this week

This week's lecture

With negative edges allowed, Dijkstra's greedy strategy breaks (a settled vertex's distance can still decrease). Bellman–Ford instead relaxes every edge $n - 1$ times, running in $O(mn)$ time, and can also detect negative cycles.

- Bellman–Ford itself dates to the 1950s. For *general* graphs with negative integer weights, no combinatorial algorithm improved meaningfully on this for decades – unlike the non-negative case, where Dijkstra plus a good priority queue gives near-linear time immediately.
- Question: is $O(mn)$ actually necessary once negative weights are allowed, the way $O(nm)$ is (conditionally) necessary for edit distance above?

The 27-year plateau

Goldberg, SIAM J. Computing, 1995 – “Scaling algorithms for the shortest paths problem”

A scaling technique (solve a coarser version of the weights, then refine) gives $O(m\sqrt{n} \log W)$ time, where W bounds the largest negative weight magnitude.

- This beats $O(mn)$ whenever n is large relative to $\log W$, and it was the best *combinatorial* bound for negative-weight SSSP for roughly 27 years.
- Near-linear time was known only for special cases (e.g. planar directed graphs) – the general case seemed stuck.
- Contrast with this week’s Dijkstra/Floyd–Warshall/DAG material: every one of those *already* runs in close to $O(m + n)$ or $O(n^3)$ for all-pairs. Negative weights, for 70 years, cost you a factor of $\sqrt{n}$ or worse.

The 2022 breakthrough: near-linear time, at last

Bernstein, Nanongkai & Wulff-Nilsen, FOCS 2022 (pp. 600–611)

“Negative-Weight Single-Source Shortest Paths in Near-linear Time.” *arXiv*: 2203.03456.

- A **randomized** algorithm solving negative-weight SSSP in $\tilde{O}(m \log W)$ time (near-linear, up to polylog factors) – matching, up to those factors, the non-negative-weight world Dijkstra already lives in.
- Core idea (very high level): combine the classical *scaling* approach with **low-diameter graph decompositions** to repeatedly reduce the problem to a bounded-diameter instance that a cheap adaptation of Dijkstra can solve, without ever paying Bellman–Ford’s full n rounds of relaxation.
- This closed a gap that had stood, in some form, since Bellman–Ford itself.

The 2026 sequel: removing the randomness

Haeupler, Jiang & Saranurak, STOC 2026 – “Deterministic Negative-Weight Shortest Paths in Nearly Linear Time via Path Covers”

arXiv: 2511.08551.

- Every near-linear-time algorithm since 2022 relied on **randomized** low-diameter decompositions. This 2026 paper gives the first fully **deterministic** nearly-linear algorithm, via a new structural primitive for directed graphs called a **path cover**.
- Running time: $\tilde{O}(m) \cdot \log(nW)$ – deterministic, no randomization anywhere, and it also detects negative cycles.
- **Small-world note:** Bernhard Haeupler is also the author (with Hershkowitz & Li) behind the “is Dijkstra universally optimal?” result cited in Week 2’s supplement – the same research group is now rewriting the negative-weight side of shortest paths too.

The bound, over 70 years

Year	Bound (negative integer weights)	Source
1950s	$O(mn)$	Bellman–Ford
1995	$O(m\sqrt{n} \log W)$	Goldberg (scaling)
2022	$\tilde{O}(m \log W)$, randomized	Bernstein–Nanongkai–Wulff-Nilsen
2026	$\tilde{O}(m) \log(nW)$, deterministic	Haeupler–Jiang–Saranurak

Perspective

Going from $O(mn)$ to near-linear is the same *qualitative* leap as going from “try every permutation” to Karatsuba in Week 1 – except here it took the research community 70 years, not one clever recursion.

Section takeaway

- ❶ $O(mn)$ Bellman–Ford is **not** optimal – unlike edit distance, there was no lower-bound wall here, just an unsolved algorithmic question, and it fell.
- ❷ The fix came from importing a tool (low-diameter decompositions) from a different corner of algorithms research, not from a cleverer relaxation order.
- ❸ As of this supplement (Aug 2026), the deterministic version is only months old – this is genuinely current research, not textbook history.

Summary

- ① **Edit distance is provably hard:** Backurs & Indyk (2015) showed $O(nm)$ is optimal for exact computation, modulo SETH – one of the cleanest fine-grained lower bounds in the field.
- ② **But only for exact, classical computation:** Boroujeni et al. (2018) show a quantum algorithm can approximate edit distance in truly subquadratic time – the wall is real, but narrower than it first looks.
- ③ **Bellman–Ford’s $O(mn)$ bound, in contrast, was never a wall:** Bernstein–Nanongkai–Wulff-Nilsen (2022) reached near-linear time, and Haeupler–Jiang–Saranurak (2026) just removed the last randomization.
- ④ **The general lesson:** “textbook optimal” always means *optimal among known techniques at the time the textbook was written* – sometimes that turns out to be a provable wall, and sometimes it is simply the frontier of what anyone had tried yet.

Looking ahead

Next week (Kleinberg & Tardos, Ch. 7) moves to **network flow** – where the story repeats once more: the textbook $O(VE^2)$ Edmonds–Karp bound is nowhere near the current frontier either.